LINEAR SEARCH

- Used to find element inside an unordered container
- Execution time proportional to the size of the container

1. Compare elements from the container with your pattern one by one.

Example: find the position of number 3 in this list / vector:

a :

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	7	5	6	2	0	1	3	5	7	9	8	2	2	3

a[0] is not 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	7	5	6	2	0	1	3	5	7	9	8	2	2	3

a[1] is not 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	7	5	6	2	0	1	3	5	7	9	8	2	2	3

a[2] is not 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	7	5	6	2	0	1	3	5	7	9	8	2	2	3

a[3] is not 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	7	5	6	2	0	1	3	5	7	9	8	2	2	3

a[4] is not 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	7	5	6	2	0	1	3	5	7	9	8	2	2	3

a[5] is not 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	7	5	6	2	0	1	3	5	7	9	8	2	2	3

a[6] is not 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	7	5	6	2	0	1	3	5	7	9	8	2	2	3

a[7] is not 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	7	5	6	2	0	1	3	5	7	9	8	2	2	3

a[8] is 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	4	7	5	6	2	0	1	3	5	7	9	8	2	2	3

→ return 8

Example: find the biggest number in a list / vector:

a:

2	1	3	7	4	2	0
0	1	2	3	4	5	6

Step 0: b = a[0]

2	1	3	7	4	2	0
0	1	2	3	4	5	6

1. a[0] is not bigger than b (2)

2	1	3	7	4	2	0
0	1	2	3	4	5	6

2. a[1] is not bigger than b (2)

2	1	3	7	4	2	0
0	1	2	3	4	5	6

3. a[2] is bigger than b (2)

2	1	3	7	4	2	0
0	1	2	3	4	5	6

→ b = a[2]

4. a[3] is bigger than b (3)

2	1	3	7	4	2	0
0	1	2	3	4	5	6

→ b = a[3]

5. a[4] is not bigger than b (7)

2	1	3	7	4	2	0
0	1	2	3	4	5	6

6. a[5] is not bigger than b (7)

2	1	3	7	4	2	0
0	1	2	3	4	5	6

7. a[6] is not bigger than b (7)

2	1	3	7	4	2	0
0	1	2	3	4	5	6

8. Return b (7)

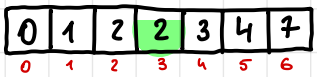
BINARY SEARCH

- Used to find an element is a sorted container
- Execution time proportional to $\log_2 N$, where N is the size of the container

1. Find median element of the container
2. If it is bigger than desired element, discard it and all bigger elements (if it is smaller, discard it and all smaller elements)
3. Repeat steps 1 & 2 until 1 element left
4. Check if it is the desired element

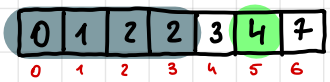
Example: We look for 3 in

0	1	2	2	3	4	7



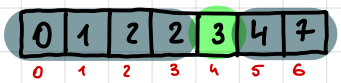
median element position: 3

2 is smaller than 3



median element position: 5

4 is bigger than 3



median element position: 4

We found the desired element at position 4

BUBBLE SORT

1. Iterate over a list comparing adjacent elements, and swapping them if needed
2. Repeat until no swap is made in a full iteration.

Examples:

1)

1 2 3 4 (ok)

1 2 3 4 (ok)

1 2 3 4 (ok)

1 2 3 4 — no swaps made — done

---

4 3 1 2

3 4 1 2

3 1 4 2

3 1 2 4 — a swap was made — repeat

---

2)

3 1 2 4

1 3 2 4

1 2 3 4 (ok)

1 2 3 4 — a swap was made — repeat

---

1 2 3 4 (ok)

1 2 3 4 (ok)

1 2 3 4 (ok)

1 2 3 4 — no swaps made — done

SELECTION SORT

1. Use linear search to find the smallest element in the list
2. Swap it with the first element
3. Repeat the process starting from the next element

Examples:

4 3 2 1 look for the smallest element within this range

1) 1 3 2 4

1 2 3 4

1 2 3 4 done

INSERTION SORT

1. Iterate on the list starting from the second element
2. Compare the current element (key) to its predecessor.
3. If the key element is smaller, compare it to the elements before. Move the greater elements one position up to make space for the swapped element

Examples:

12 11 13 5 6

11 12 13 5 6 ok

11 12 13 5 6

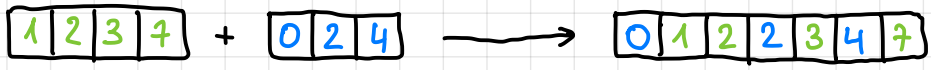
5 11 12 13 6

5 6 11 12 13

MERGE SORT

Merge function

Function that takes two sorted lists A and B as arguments, and put their elements in the list C, keeping it sorted



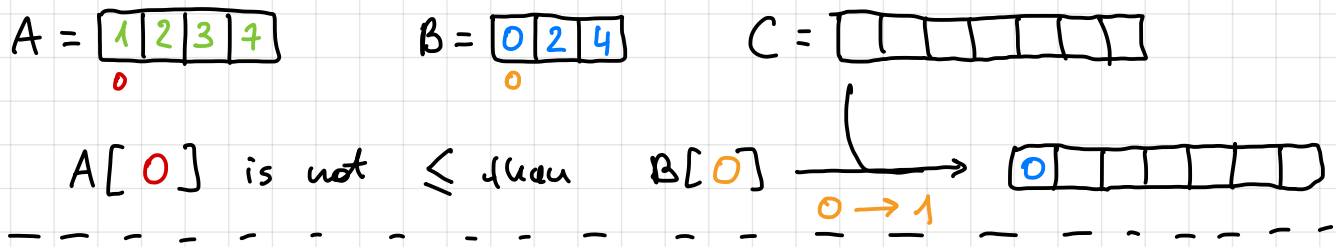
Let's define: $i = 0$, $j = 0$, n = size of A, m = size of B

Then, as long as $i < n$ or $j < m$, repeat:

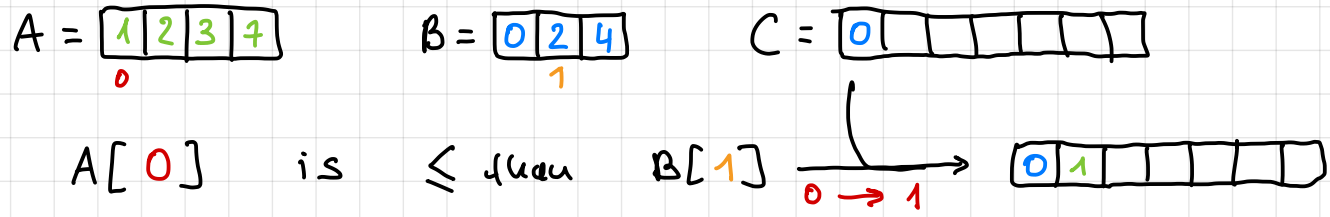
1. if $A[i] \leq B[j]$, put $A[i]$ to C and increase i by one
2. Else, put $B[j]$ to C and increase j by one

Example:

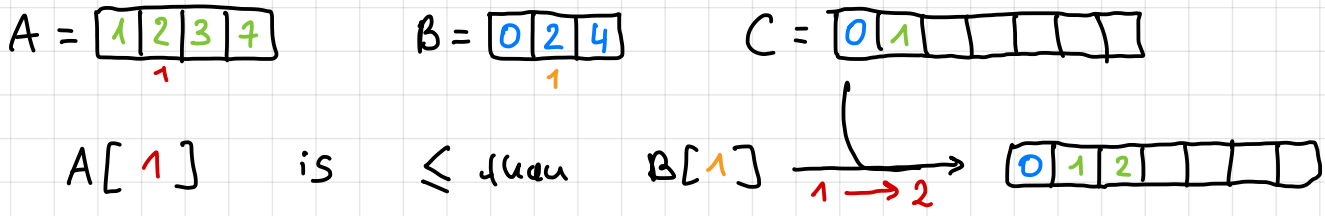
1)



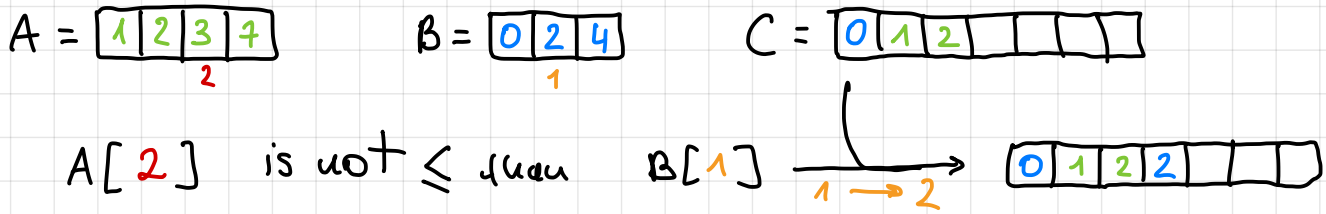
2)



3)



4)



etc

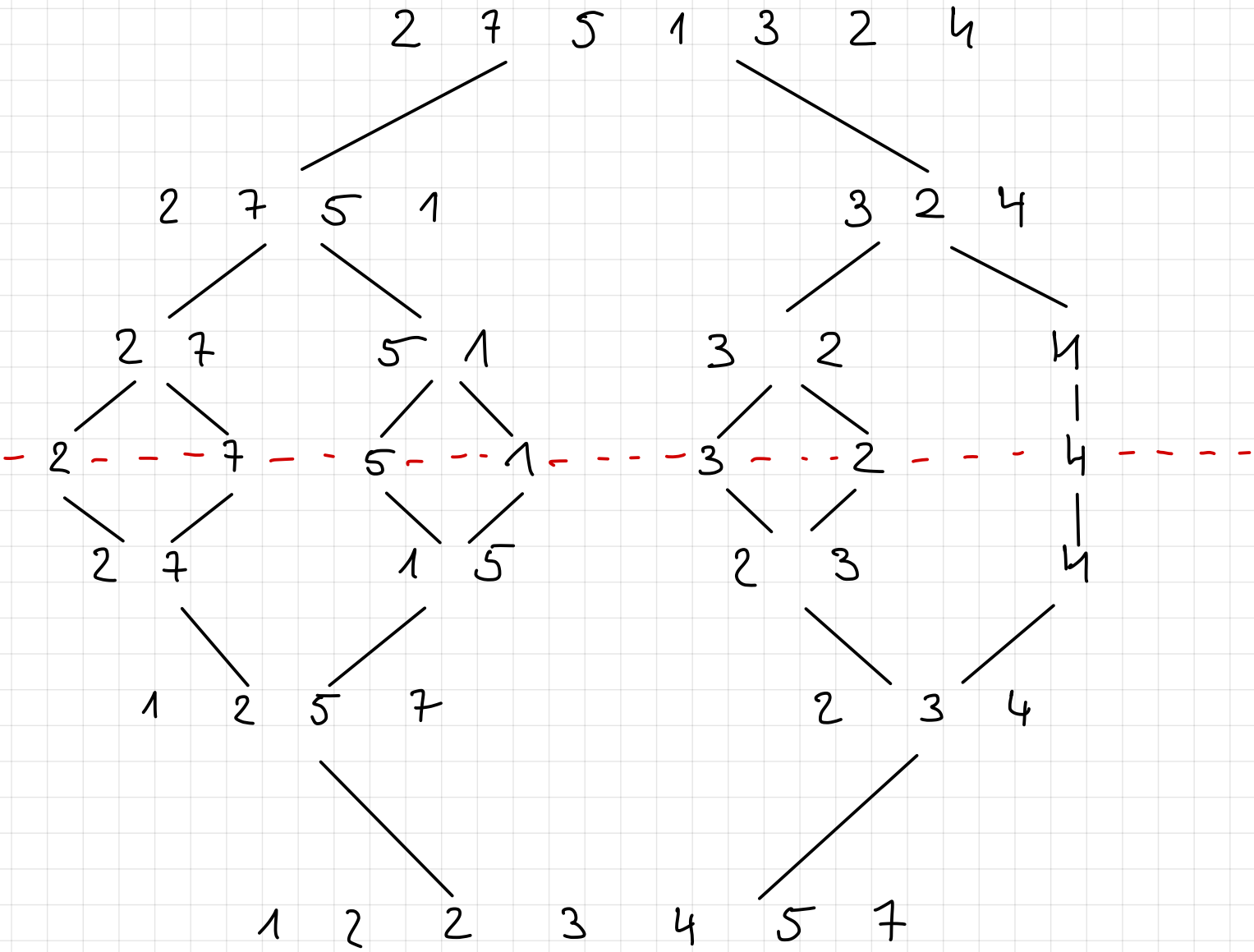
Recursive merge sort

To sort the list A:

a) if it has more than 1 element, let's split it into two lists: B and C, sort them using merge sort, then merge them, and return merged list

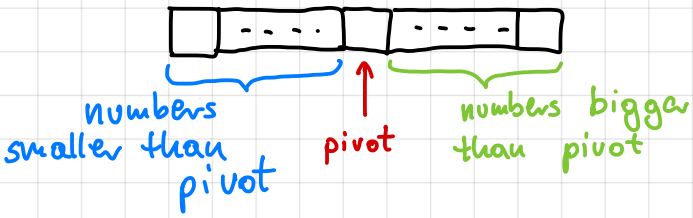
b) if it has one element, just return list A

Example:



QUICK SORT

1) Choose an arbitrary number (for simplicity, let's choose the last number from the list). It's called pivot
2) Go through a list, move all numbers smaller than pivot to the left, and bigger to the right



3) Repeat the procedure for blue and green parts of the list separately
4) Keep reapeating until there is only one element to process

Instruction for step 2

Let $i = 0$, $j = -1$, $n =$ size of A

Then, as long as $i < n$:

1. If $A[i] >$ pivot, increase i by 1.
2. Else, increase j by 1, then swap $A[i]$ with $A[j]$, then increase i by 1.

Example:

$A =$ | 2 | 1 | 3 | 7 | 0 | 4 | 2 | $\quad n = 7$

pivot

1)

2	1	3	7	0	4	2
-1	0					

A[0] is not > 2:

$-1 \longrightarrow 0$ A[0] $\longleftrightarrow$ A[0]

$0 \longrightarrow 1$

---

2)

2	1	3	7	0	4	2
0	1					

A[1] is not > 2:

$0 \longrightarrow 1$ A[1] $\longleftrightarrow$ A[1]

$1 \longrightarrow 2$

---

3)

2	1	3	7	0	4	2
1	2					

A[2] is > 2:

$2 \longrightarrow 3$

---

4)

2	1	3	7	0	4	2
1		3				

A[3] is > 2:

$3 \longrightarrow 4$

---

5)

2	1	3	7	0	4	2
1				4		

A[4] is not > 2:

$1 \longrightarrow 2$ A[4] $\longleftrightarrow$ A[2]

$4 \longrightarrow 5$

---

6)

2	1	0	7	3	4	2
		2			5	

A[5] is > 2;

$5 \longrightarrow 6$

---

7)

2	1	0	7	3	4	2
	2					6

A[6] is not > 2 :

2 ⟶ 3 A[6] ⟷ A[3]

6 ⟶ 7

---

8)

2	1	0	2	3	4	7

7 is not < n : stop